



Tribhuvan University
Faculty of Humanities and Social Sciences

A
PROJECT PROPOSAL
ON
NEPAL BUS TICKET BOOKING SYSTEM
(Course Code: CAPJ356)



Submitted to Department of Information Technology
Kabhre Multiple Campus

*A Proposal submitted for the partial fulfillment of requirements for the degree of
Bachelor in Computer Application (Sixth Semester)*

Submitted by
Nischal Shrestha
Registration Number: 6-2-244-15-2023
Roll No: 24402165

Under Supervision of
[Supervisor Name]

October, 2026

Tribhuvan University
Faculty of Humanities and Social Sciences
Kabhre Multiple Campus

Supervisor's Recommendation

I hereby recommend that this project proposal prepared under my supervision by **Nischal Shrestha** entitled “**NEPAL BUS TICKET BOOKING SYSTEM**” in partial fulfillment of the requirements for the degree of Bachelor of Computer Application is recommended for the proposal defense.

.....
Signature
[Supervisor Name]
SUPERVISOR
[Academic Designation]
Department of Information Technology
Kabhre Multiple Campus, Banepa-Budol, Nepal

Letter of Approval

This project proposal entitled “**NEPAL BUS TICKET BOOKING SYSTEM**” by **Nischal Shrestha** as a partial fulfillment of the requirements for the degree of Bachelor in Computer Application (Sixth Semester) has been examined by us and is accepted.

Panel of Examiners:

Name

Signature

Date

1. HOD [HOD Name]

2.

3.

4.

Abstract

Long-distance bus ticketing in Nepal still depends largely on physical counters, phone calls, and local agents. This causes problems for passengers, who cannot easily check seat availability or compare fares, and for operators, who manage seat records manually and face double booking and disputes. This project proposes the Nepal Bus Ticket Booking System, a web-based application that lets passengers search for buses by origin, destination, and travel date, choose seats from a visual seat map, and book tickets online. Bus operators can manage routes, buses, schedules, and fares, and administrators can oversee users, bookings, and reports.

The system prevents double booking through a temporary seat-locking mechanism. It uses sorting and searching algorithms to present results by price or departure time, a fare calculation module that supports special pricing rules, and reports on bookings, revenue, and seat occupancy. Online payment is integrated through eSewa, and a booking is confirmed only after the payment is verified. The system will be developed as a responsive web application using Next.js, NestJS, PostgreSQL, and Tailwind CSS.

Keywords: bus ticket booking, seat locking, web application, Nepal, NestJS, PostgreSQL

Acknowledgement

I would like to express my sincere gratitude to my supervisor, [Supervisor Name], for the guidance, suggestions, and encouragement given throughout the preparation of this project proposal.

I am thankful to [HOD Name], Head of the Department of Information Technology, and to Kabhre Multiple Campus for giving me the opportunity to work on this project. I also thank all the faculty members of the department for the knowledge and support they have provided during my studies.

Finally, I would like to thank my family and friends for their continuous support and motivation.

Nischal Shrestha
Registration Number: 6-2-244-15-2023
Roll No: 24402165
October, 2026

Table of Contents

ABSTRACT	i
ACKNOWLEDGEMENT	ii
LIST OF ABBREVIATIONS	iv
LIST OF TABLES	v
CHAPTER 1: INTRODUCTION	1
1.1 Introduction	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope and Limitation	2
1.4.1 Scope	2
1.4.2 Limitations	3
1.5 Development Methodology	3
1.6 Report Organization	3
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW	5
2.1 Background Study	5
2.1.1 Study of Existing System	5
2.1.2 Key Concepts	5
2.2 Literature Review	6
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN	8
3.1 System Analysis	8
3.1.1 Requirement Analysis	8
3.1.2 Feasibility Analysis	11
3.1.3 Object Modelling	11
3.1.4 Dynamic Modelling	12
3.2 High-Level Design	16
3.3 Algorithm Details	16
CHAPTER 4: IMPLEMENTATION AND TESTING	19
4.1 Implementation	19
4.1.1 Tools Used	19
4.1.2 Implementation Details of Modules	19
4.2 Testing Plan	19
4.3 Project Schedule	20
CHAPTER 5: EXPECTED OUTCOME AND FUTURE RECOMMENDATIONS	21
5.1 Expected Outcome	21
5.2 Future Recommendations	21
REFERENCES	22

List of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
CASE	Computer-Aided Software Engineering
CSS	Cascading Style Sheets
GPS	Global Positioning System
HOD	Head of Department
HTML	HyperText Markup Language
IEEE	Institute of Electrical and Electronics Engineers
PHP	PHP: Hypertext Preprocessor
SMS	Short Message Service
SQL	Structured Query Language
XHTML	Extensible HyperText Markup Language

List of Tables

2.1	Comparison of the manual system, existing platforms, and the proposed system	6
-----	--	---

CHAPTER 1: INTRODUCTION

1.1 Introduction

Road transport is the main means of long-distance travel in Nepal, and buses connect cities such as Kathmandu, Pokhara, Chitwan, and Birgunj to the rest of the country. Despite this, ticket booking for many bus services still depends on physical counters, phone calls, or local agents. Passengers often have to visit the counter in person to check seat availability, compare prices, or confirm a booking, and operators must manage seat records manually.

The Nepal Bus Ticket Booking System is a web-based application that allows passengers to search for buses, view available seats, and reserve tickets online. Passengers can search by origin, destination, and travel date, select seats from a visual seat map, and receive a booking confirmation. Bus operators can manage their routes, buses, schedules, and fares, while administrators oversee users, bookings, and reports.

The system focuses on solving the problem of double booking through a seat-locking mechanism, which holds a seat temporarily while a passenger completes the booking. It uses sorting and searching algorithms to present results by price or departure time, and a fare calculation module for pricing rules. The system will be developed as a responsive, mobile-friendly web application using Next.js, NestJS, PostgreSQL, and Tailwind CSS. Online payment will be integrated through eSewa.

1.2 Problem Statement

Many long-distance bus services in Nepal still rely on manual ticketing. Passengers must visit counters or contact agents to find out which buses are running, whether seats are available, and what the fare is. Comparing the options of different operators on a route takes time, and a passenger often cannot be sure a seat is confirmed until they reach the counter.

Manual booking also creates problems for operators. Seat records are kept on paper or in separate registers, which makes errors likely. When two agents or counters sell the same seat, double booking occurs and leads to disputes at boarding time. Cancellations, fare changes during festival seasons, and sales reports are hard to track without a central system.

Because of these issues, there is a need for a system that shows live seat availability, prevents the same seat from being booked twice, and gives operators one place to manage routes, schedules, fares, and bookings. This project addresses that need by developing a web-based bus ticket booking system tailored to the Nepali context.

1.3 Objectives

The main objective of this project is to develop a web-based bus ticket booking system that makes seat reservation easier and more reliable for passengers and operators in Nepal.

The specific objectives are:

1. To develop a search module that lets passengers find buses by origin, destination, and travel date, and sort the results by price or departure time.
2. To implement a seat selection and booking module that shows live seat availability and prevents double booking through a temporary seat-locking mechanism.
3. To provide role-based access for passengers, bus operators, and administrators, with secure authentication.
4. To develop a fare calculation module that supports route-based pricing and special pricing rules, such as peak-season fares.
5. To implement a booking and cancellation workflow with online payment through eSewa, confirming a booking only after the payment is verified.
6. To generate reports for operators and administrators, such as total bookings, revenue, and seat occupancy.

1.4 Scope and Limitation

1.4.1 Scope

The scope of this project covers the design and development of a web-based bus ticket booking system for long-distance routes in Nepal. The system is a responsive web application for desktop and mobile browsers and includes the following:

1. Three user roles (passengers, bus operators, and administrators) with registration, login, secure password storage, and role-based access.
2. Management of routes, buses, schedules, and fares by operators and administrators.
3. Searching for buses by origin, destination, and travel date, with results sorted by price or departure time.
4. A visual seat map showing available and booked seats, with temporary seat locking to prevent double booking.
5. Fare calculation with special rules such as peak-season fares, along with booking, cancellation, and booking history with online payment through eSewa.
6. Reports for operators and administrators, such as total bookings, revenue, and seat occupancy.

1.4.2 Limitations

The system has the following limitations:

1. Payment works only in the eSewa test (sandbox) environment with test accounts, so no real money is processed. Moving to live payments needs a registered merchant account and is left for future work.
2. The system runs on sample data. It is not connected to real bus operators, and the routes and fares are for demonstration only.
3. It does not provide live bus tracking or GPS location.
4. It is a web application only. A native mobile app is not developed.
5. It is not tested for large-scale traffic and is designed for a limited number of users.
6. Notifications such as SMS or email are not included in the current version.

1.5 Development Methodology

The system is developed using the iterative incremental model of the software development life cycle. Instead of building everything at once, the system is divided into increments. Each increment goes through requirements, design, implementation, and testing, and ends with a working part of the system that is reviewed with the supervisor before the next increment starts. This model suits a single developer with weekly supervisor meetings, because problems in the design are found early, while they are still cheap to fix.

The planned increments are:

1. Authentication and role-based access for passengers, operators, and administrators.
2. Management of buses, routes, schedules, and fares.
3. Bus search and the seat map, with sorting of results.
4. Seat locking and booking, including cancellation.
5. Online payment through eSewa, with payment verification.
6. Special fare rules and reports.

Each increment is tested when it is finished, and a final round of system testing and documentation follows the last increment.

1.6 Report Organization

This proposal is organized into five chapters. Chapter 1 introduces the project, the problem, the objectives, the scope and limitations, and the development methodology. Chapter 2

presents the background study, including a study of the existing system, and the literature review. Chapter 3 covers the system analysis and design: the requirements, the feasibility analysis, the object-oriented models, the high-level design, and the algorithms. Chapter 4 describes the planned implementation and testing, which covers the tools, the modules, the testing approach, and the project schedule. Chapter 5 gives the expected outcome and the future recommendations. The references are listed at the end.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

2.1.1 Study of Existing System

Manual system. In the traditional system, a passenger books a long-distance bus ticket by visiting the operator's counter, calling the office, or going through a local agent. The staff check a paper register or a seat chart, tell the passenger which seats are free, take the payment in cash, and write the booking by hand. Each counter or agent keeps its own record, so seat information is not shared in real time. This leads to double booking, long queues, errors in the records, and difficulty in producing sales reports.

Existing online platforms. Several online platforms already offer bus ticket booking in Nepal. BusSewa describes itself as a real-time booking platform that lets passengers search routes, compare fares, choose seats, and receive e-tickets, with payment through wallets, banks, and cards [1]. GoMyGo allows several passengers to be booked in one transaction, sends confirmations by email and SMS, and follows each operator's own cancellation policy [2]. Okay Journey offers an interactive seat map and a choice of bus class [3]. Trip Turbo lets users filter buses by price and departure time, pick window or aisle seats from a seat map, and receive an e-ticket by email and SMS [4].

Comparison. Table 2.1 compares the manual system, the existing online platforms, and the proposed system.

Summary. Existing platforms have solved the main problems of the manual system for the operators they work with. This project does not aim to replace them. Its purpose is to build the core booking logic as a complete system with its own modules: seat locking to prevent double booking, fare rules, sorting and searching of results, role-based management for passengers, operators, and administrators, and reports. The modules are written for this project and not taken from plugins or third-party services, so they can be studied, tested, and extended.

2.1.2 Key Concepts

[To be written]

Table 2.1: Comparison of the manual system, existing platforms, and the proposed system

Feature	Manual system	Existing online platforms	Proposed system
Seat availability	Checked at the counter or by phone	Real-time	Real-time
Seat selection	Done by staff, limited	Interactive seat map	Interactive seat map
Booking record	Paper registers	Stored in a database	Stored in a database
Payment	Cash at the counter	Wallets, banks, cards	eSewa (sandbox test mode)
Notifications	None	Email and SMS	Not included
Operator reports	Hard to prepare	Offered through the platform	Bookings, revenue, occupancy
Scope	Single counter	Commercial, many operators	Academic project, sample data

2.2 Literature Review

Several studies have looked at online bus ticket reservation, and their basic features are very similar.

Oloyede et al. developed a web-based bus reservation system for a transport service in Nigeria using XHTML, PHP, SQL, Ajax, CSS, and JavaScript [5]. Their system lets users check seat availability, reserve a ticket, and cancel it, and it gives administrators the means to verify users, add vehicles, and generate reports [5]. They report that manual reservation handled separately at each branch causes delays and risks of duplicate bookings [5]. Their work, however, does not describe how two simultaneous requests for the same seat are handled.

Chavan et al. describe a computerized reservation system to replace manual record keeping, which they say caused errors in data entry and disputes with customers [6]. The system keeps passenger details, seat availability, price per seat, and bill generation, and is built with HTML, PHP, CSS, JavaScript, and MySQL [6]. It is a short study of a basic design and does not cover fare rules or concurrent bookings.

Venkat and Raju present a bus ticketing application built with Django and MySQL [7]. Users search by source, destination, and date, and seat availability is calculated in real time from the booking records [7]. The system also offers fare calculation, booking history, cancellation of future tickets, and an administrative module for routes and bookings [7]. The abstract states that data consistency is maintained for multiple users but does not describe the mechanism used [7].

The seat-conflict problem is treated in more detail in ticketing research for high traffic. Zhang et al. studied a railway ticketing system and note that under heavy load the same ticket can be sold more than once [8]. Their design temporarily locks the seat when an order is created and deducts it only after payment succeeds, and uses an atomic token-deduction step in Redis to prevent overselling [8]. Their system uses microservices and several middleware components, which is far larger than what a single-developer project needs [8].

Summary of the review. The reviewed systems agree on a common core: searching buses, checking seats, booking, cancelling, and administrator management [5, 6, 7]. The simpler systems do not explain how double booking is prevented, while the research on high-traffic ticketing shows that it is the central problem [8]. This project keeps the common core and adds an explicit temporary seat-locking mechanism, role-based management for passengers, operators, and administrators, and fare rules. It also takes a lighter approach than the microservice design in [8], which suits a single-developer project on a single server.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

3.1.1 Requirement Analysis

Figure 3.1 illustrates the use case diagram for the Nepal Bus Ticket Booking System. It defines the system boundary and models the functional requirements by showing how external actors interact with the primary features of the system.

The diagram identifies four primary actors and their interactions within the system:

- **Passenger:** Interacts with the system to search for bus trips, select seats with temporary holds, initiate online payments, view booking confirmations, and request booking cancellations.
- **Bus Operator:** Configures and manages fleet details, seat layouts, routes, travel schedules, dynamic peak fare rules, and views booking reports for their specific fleet.
- **Administrator:** Governs system-level operations, including reviewing and approving new bus operators, managing user accounts, and accessing system-wide revenue and occupancy analytics.
- **eSewa Gateway:** Acts as an external payment service integrated into the *Make On-line Payment* use case via an «include» relationship to verify transactions securely.

i. Functional requirements

Passenger

1. Register, log in, and log out securely.
2. Search for buses by origin, destination, and travel date, and sort the results by price or departure time.
3. View the seat map of a bus and select one or more available seats.
4. Enter passenger details and pay online through eSewa.
5. View the booking confirmation and the booking history.
6. Cancel a booking according to the cancellation rule set in the system.

Bus operator

1. Log in to a separate operator account.

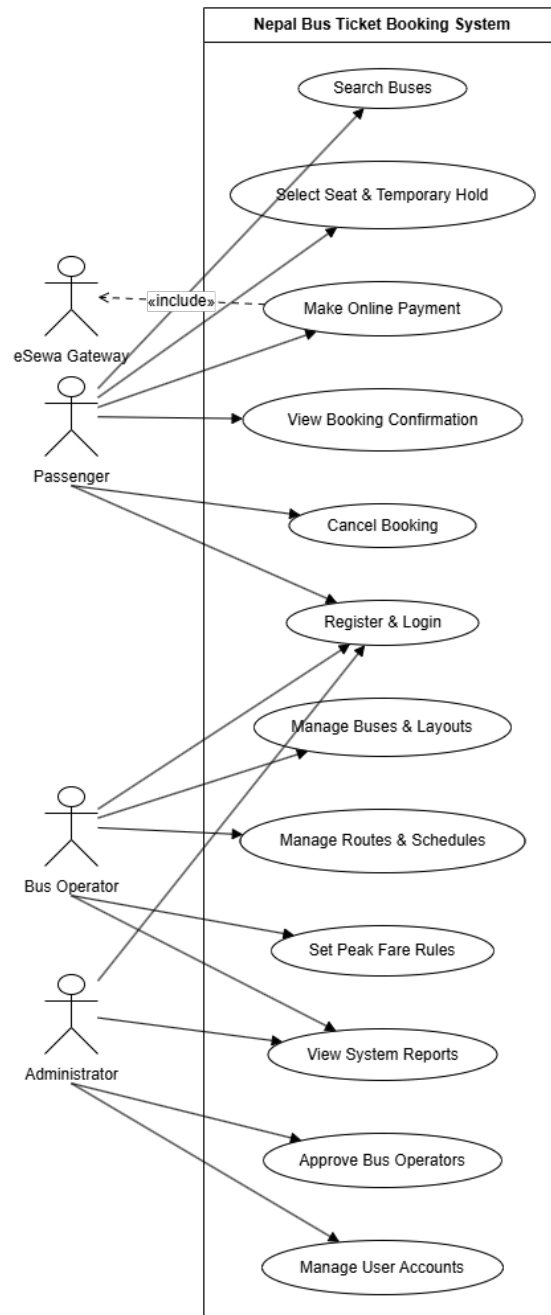


Figure 3.1: Use case diagram of the Nepal bus ticket booking system

2. Add and manage buses, including the bus type and seat layout.
3. Add and manage routes, schedules, and fares, including special pricing such as peak-season fares.
4. View the bookings made on their own buses.
5. Cancel a trip when needed.
6. View reports for their own buses, such as bookings, revenue, and seat occupancy.

Administrator

1. Log in to an administrator account.
2. Approve and manage bus operators.
3. Manage user accounts, including blocking an account.
4. View all bookings across the system.
5. View overall reports for all operators.

System

1. Prevent the same seat from being booked by two passengers, holding a selected seat temporarily while the booking is completed.
2. Calculate the fare from the route price and the pricing rules.
3. Confirm a booking only after the payment has been verified with eSewa.
4. Allow each role to access only its own features.

ii. Non-functional requirements

1. *Security*: passwords are stored in hashed form, and only logged-in users can book or manage data.
2. *Data integrity*: a booking, its seats, and its payment status are saved together or not at all.
3. *Usability*: the interface is simple, responsive, and works on mobile and desktop browsers.
4. *Performance*: search results and seat maps load in a reasonable time for the expected number of users.
5. *Maintainability*: the code is organized in separate modules, so features can be changed or extended.
6. *Compatibility*: the system works on current versions of common web browsers.

3.1.2 Feasibility Analysis

A feasibility study checks whether the proposed system can be built and used with the available skills, time, and resources. It is considered under three headings.

3.1.2.1 Technical Feasibility

The system uses open-source technologies that the developer already knows: Next.js and Tailwind CSS for the interface, NestJS for the server, both written in TypeScript, and PostgreSQL for the database. PostgreSQL's transactions and row-level locking support the seat-locking mechanism that prevents double booking. Online payment is integrated through the eSewa ePay sandbox, which provides test accounts and documented request and verification steps. Git and GitHub are used for version control. All of these are well documented and run on an ordinary computer. The main technical risk is getting the seat locking right when several users request the same seat at once, which will be reduced by using database transactions with row-level locks and by testing the booking flow with simultaneous requests. The project is therefore technically feasible.

3.1.2.2 Operational Feasibility

The system replaces counter visits and phone calls with web pages that follow a familiar flow: search for a bus, choose a seat, and book. Passengers only need a web browser and an internet connection, and the responsive design lets them use a phone or a computer. Operators and administrators get separate dashboards to manage buses, routes, fares, and bookings, which replaces paper registers. No special training is needed beyond a short introduction to the dashboards. The system runs on sample data, so its use by real operators is not part of this project. The system is operationally feasible.

3.1.2.3 Economic Feasibility

All the tools used are free, including Node.js, Next.js, NestJS, PostgreSQL, Tailwind CSS, Git, GitHub, and the code editor, so there are no licensing costs. Payment uses the free eSewa sandbox with test accounts, so no payment fees or merchant account are needed during development. The system can be developed and demonstrated on the developer's own computer. The main cost is the developer's time. The expected benefits are less manual work, shorter queues, and fewer record errors for operators, so the system is economically feasible.

3.1.3 Object Modelling

The system is designed with the object-oriented approach. Figure 3.2 shows the class diagram, which describes the main classes of the system, their attributes and operations, and

the relationships between them.

Users. *User* is an abstract class that holds the common account data and the register, login, and logout operations. *Passenger*, *Operator*, and *Administrator* inherit from it and add the operations of their own role. A passenger searches for buses and makes and cancels bookings. An operator manages buses, trips, and fare rules and views reports. An administrator approves operators, manages users, and views all bookings and system reports.

Buses and trips. An *Operator* owns many *Bus* objects, and each bus is composed of its *Seat* objects. A *Route* and a *Bus* together define a *Trip*, which has a departure time and a base fare. *FareRule* lets an operator set special pricing, such as peak-season fares, for a route.

Bookings and payments. A *Passenger* makes *Booking* objects for a *Trip*. A booking includes one or more *BookingSeat* objects, each linking the booking to a seat and to the passenger's name. A booking can have *Payment* objects, which store the eSewa transaction and its status. The *Booking* class also carries the seat-locking operations: `holdSeats()` holds the selected seats for a limited time, and `releaseHold()` releases them when the time runs out without payment.

Object diagram. Figure 3.3 shows an object diagram, which is a snapshot of the system with example objects. In it, the passenger Sita Sharma has made a confirmed booking for a Kathmandu to Pokhara trip operated by Sample Travels. The booking holds seat A1 of the bus. Its total of 1440 is the base fare of 1200 plus the 20 percent peak-season rule, and it is paid by one completed eSewa payment. The names and values are sample data.

3.1.4 Dynamic Modelling

Dynamic modelling shows how the objects of the system change over time and how they work together. State diagrams show the states that one object passes through, and a sequence diagram shows the messages that are exchanged between the parts of the system during a scenario. This section gives a state diagram for a booking, a state diagram for a seat, and a sequence diagram for booking and paying for seats.

State diagram of a booking. Figure 3.4 shows the states of a *Booking*. When a passenger selects seats, `holdSeats()` creates the booking in the *Held* state, and the seats stay held until `holdExpiresAt`. A failed payment leaves the booking in the *Held* state, so the passenger can try again. When the server verifies the payment, `confirm()` moves the booking to *Confirmed*. If the hold time ends before the payment is completed, `releaseHold()` moves the booking to *Expired* and releases the seats. A passenger can cancel a held booking, and can cancel a confirmed booking only if the cancellation rule allows it. In both cases `cancel()` moves the booking to *Cancelled* and releases the seats. *Expired* and *Cancelled* are end states.

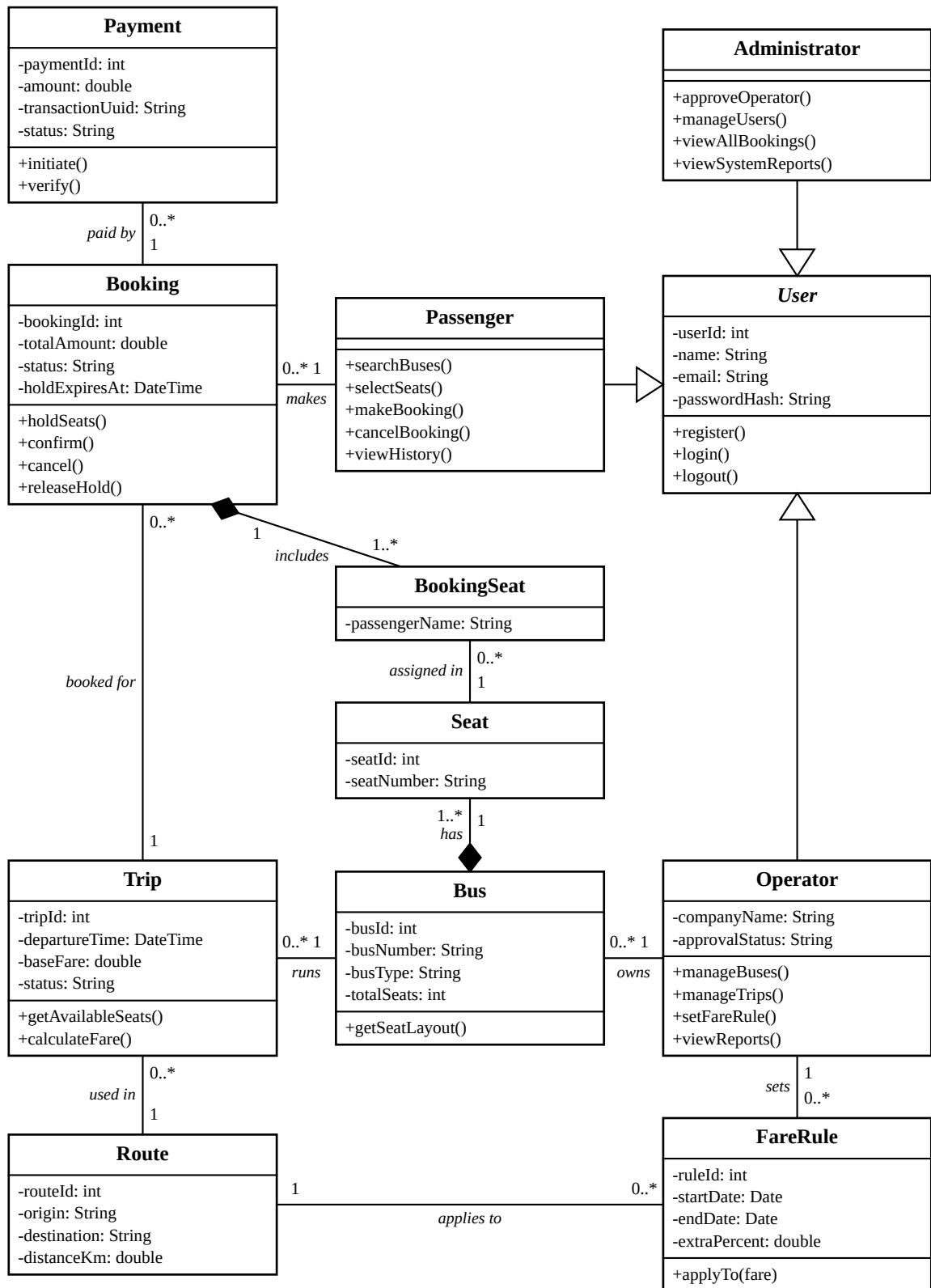


Figure 3.2: Class diagram of the bus ticket booking system

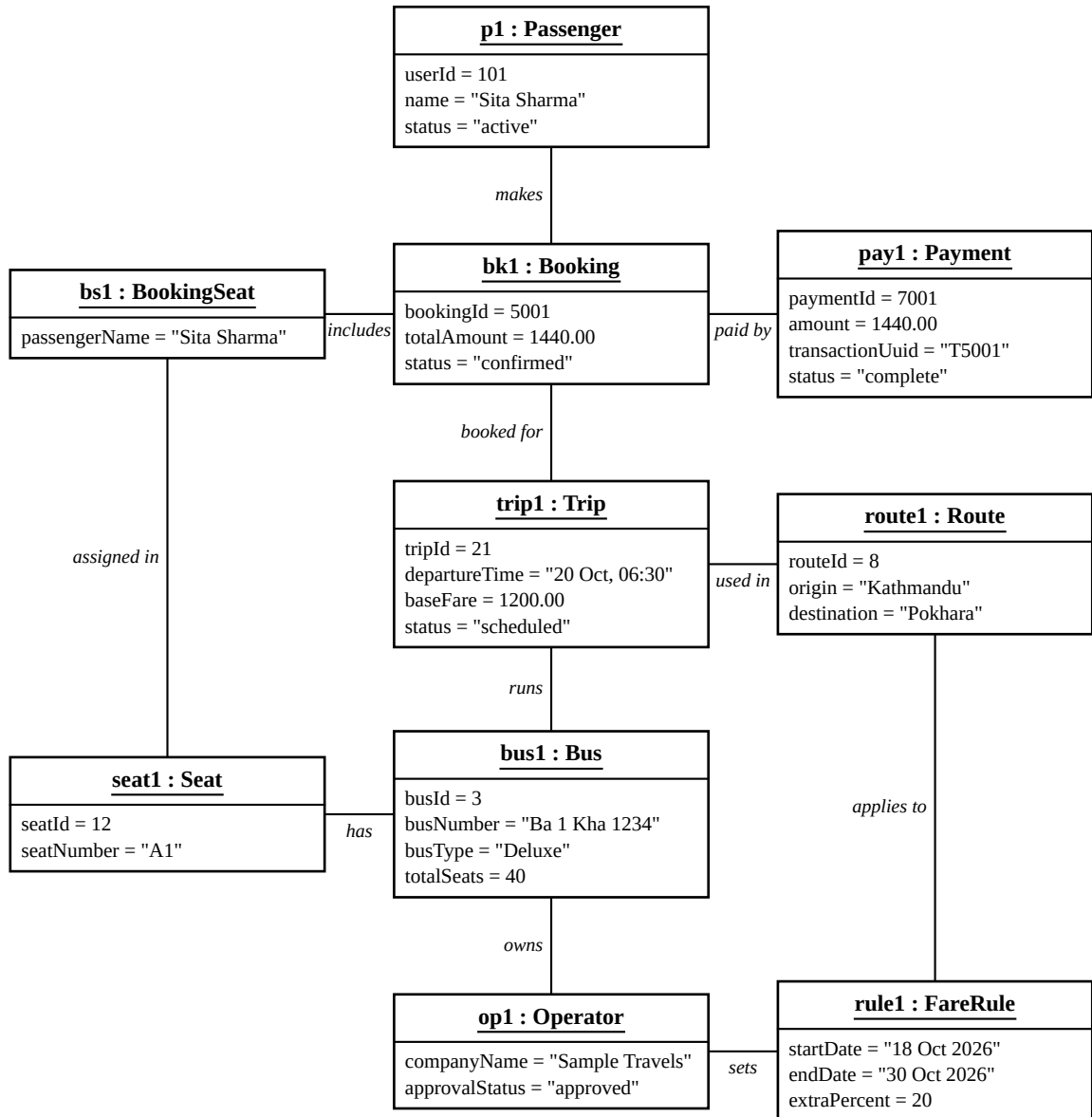


Figure 3.3: Object diagram of a confirmed booking

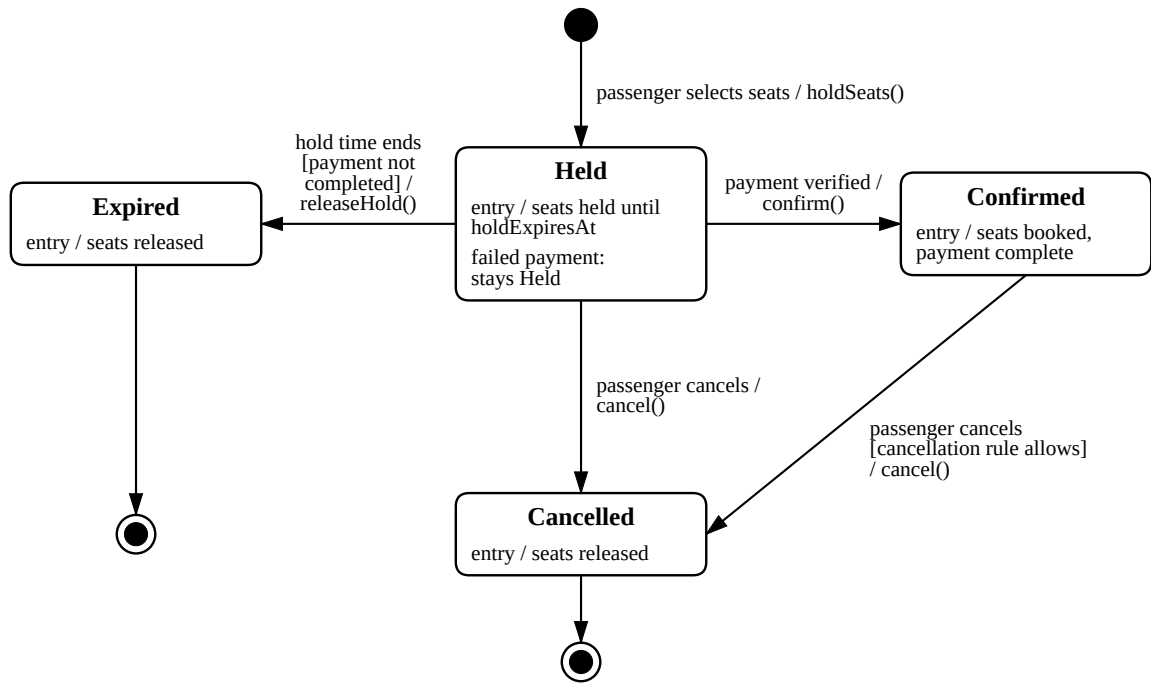


Figure 3.4: State diagram of a booking

State diagram of a seat. Figure 3.5 shows the state of one seat on one trip. A seat is *Available* when the trip is scheduled. It becomes *Held* when a passenger holds it with `holdSeats()`, and *Booked* when the booking is confirmed. A held seat goes back to *Available* when the hold ends with `releaseHold()` or when the booking is cancelled before payment, and a booked seat goes back to *Available* when the booking is cancelled. The database allows only one passenger to hold a seat at a time, so a seat that is *Held* or *Booked* cannot be held by another passenger.

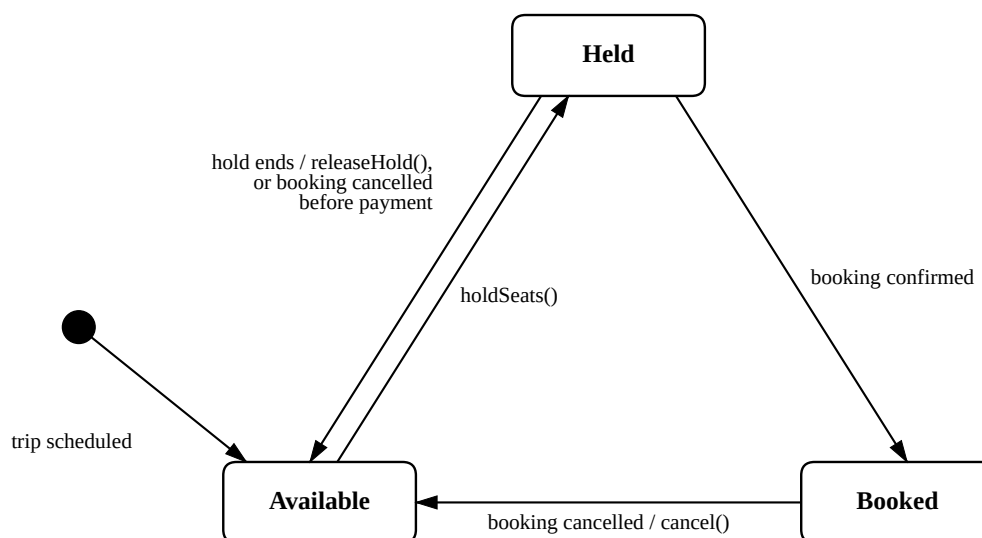


Figure 3.5: State diagram of a seat

Sequence diagram of booking and payment. Figure 3.6 shows the messages exchanged

between the passenger, the web application (Next.js), the server (NestJS), the database (PostgreSQL), and eSewa. After the passenger selects a trip and seats, the server calls `holdSeats()`. In one database transaction it locks the seat rows and checks that the seats are free. If they are free, the server saves the booking with the status held and the hold expiry time, and the passenger is shown the payment page. If the seats have already been taken, the passenger is asked to choose other seats.

When the passenger confirms, the server saves a payment with the status pending and returns a signed payment request. The web application redirects the passenger to eSewa, where the passenger approves the payment, and eSewa redirects the passenger back with the payment result. The web application sends this result to the server, and the server does not trust it directly. It asks eSewa for the payment status. Only if the status is complete does the server update the payment, confirm the booking, and mark the seats as booked. Otherwise the payment is marked as failed and the booking stays held, so the passenger can retry until the hold ends. If the hold time ends without payment, the server calls `releaseHold()` and the seats become available again.

3.2 High-Level Design

Figure 3.7 shows the high-level design of the system. It has three layers and one external service. The web application, built with Next.js and Tailwind CSS, is what passengers, operators, and administrators use in a web browser. It sends requests to the server and shows the responses. The server, built with NestJS, holds the business logic: login and role-based access, bus, route, and trip management, search, booking with seat locking, fare rules, payment, and reports. It reads and writes all data in a PostgreSQL database, which supports the transactions and row-level locks that the seat-locking mechanism needs.

Payment is handled by eSewa, which is outside the system. The server sends a payment request to eSewa and, after the passenger pays, asks eSewa for the payment status before it confirms a booking. The sequence diagram in Figure 3.6 shows how these parts work together during a booking.

3.3 Algorithm Details

[To be written]

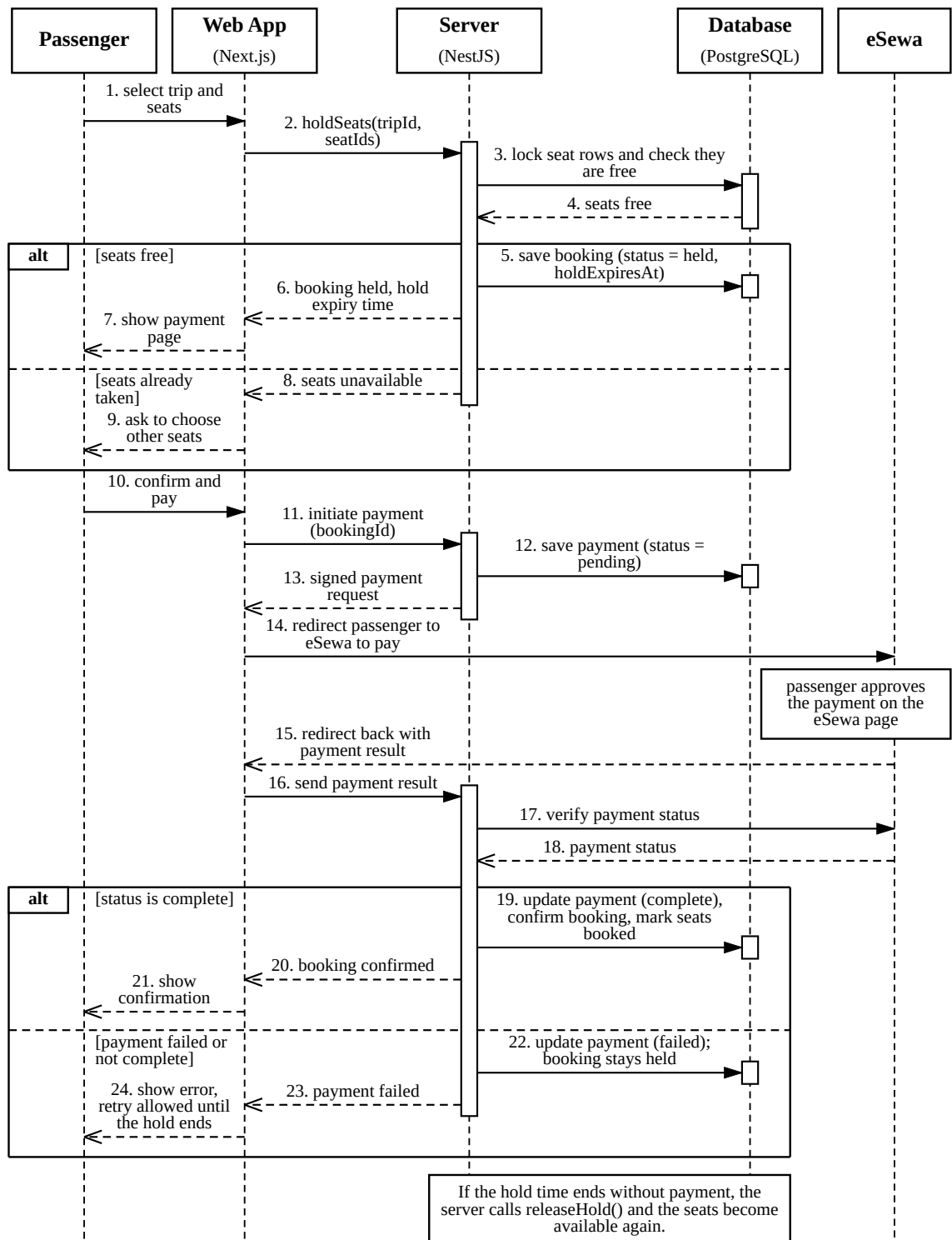


Figure 3.6: Sequence diagram of booking and payment

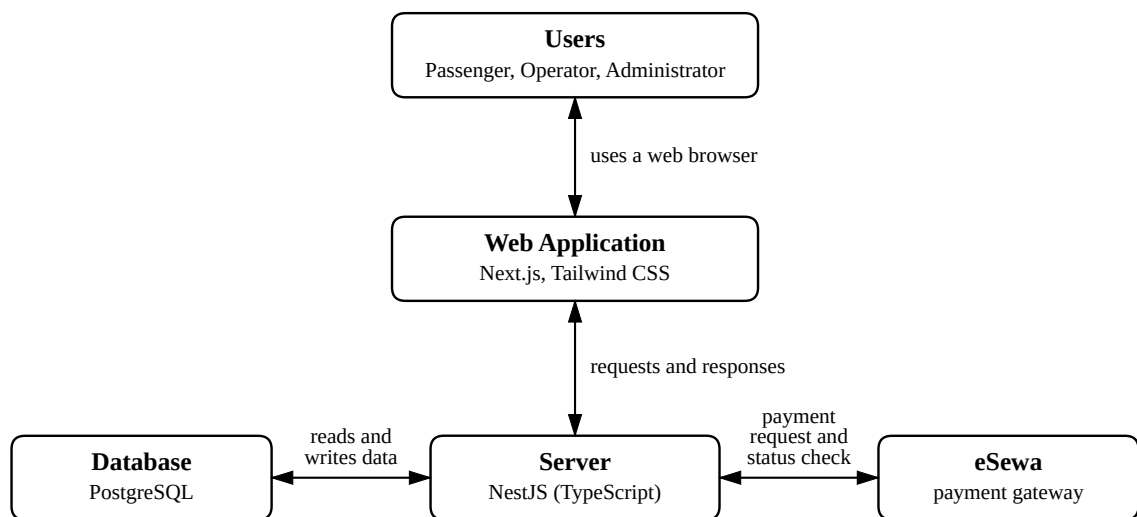


Figure 3.7: High-level architecture of the system

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Implementation

4.1.1 Tools Used

The following tools are used to develop and manage the project.

CASE and development tools. Visual Studio Code is the code editor. The state diagrams, the sequence diagram, and the architecture figure are drawn with Python (Matplotlib) scripts. Claude is used as an AI assistant for design discussion, drafting, and review. Notion is used for project tracking and management. Git is used for version control, and the source code is hosted on GitHub.

Programming languages and frameworks. The system is written in TypeScript on Node.js. Next.js and Tailwind CSS are used for the front end, and NestJS is used for the back end.

Database. PostgreSQL is used as the database.

4.1.2 Implementation Details of Modules

The system is divided into the following modules, which follow the increments described in Section 1.5.

1. *Authentication and role management:* registration, login, and role-based access for passengers, operators, and administrators.
2. *Bus, route, and trip management:* buses with seat layouts, routes, schedules, and fares.
3. *Search and seat map:* searching buses by origin, destination, and date, with results sorted by price or departure time, and a seat map for each trip.
4. *Booking and seat locking:* temporary seat holds, booking, and cancellation.
5. *Payment:* online payment through eSewa, with verification before a booking is confirmed.
6. *Fare rules and reports:* special fare rules and reports on bookings, revenue, and seat occupancy.

4.2 Testing Plan

The system will be tested as each increment is completed. The detailed test cases will be written during development and reported in the final report.

Unit testing. Unit tests will cover the functions of each module, such as fare calculation, seat holding and release, sorting of search results, and password hashing.

System testing. System tests will follow the complete booking flow, from search to confirmed booking, including payment verification and cancellation. They will include a test with simultaneous requests for the same seat, to confirm that double booking does not occur.

4.3 Project Schedule

[To be written]

CHAPTER 5: EXPECTED OUTCOME AND FUTURE RECOMMENDATIONS

5.1 Expected Outcome

At the end of the project, the following are expected to be delivered:

1. A working web-based bus ticket booking system with separate access for passengers, bus operators, and administrators, protected by secure login.
2. A search module that finds buses by origin, destination, and travel date, and sorts the results by price or departure time.
3. A seat selection and booking module with a live seat map and a temporary seat-locking mechanism that prevents double booking, tested with simultaneous booking requests.
4. A fare calculation module that supports route-based pricing and special pricing rules, such as peak-season fares.
5. A booking, cancellation, and booking-history workflow with online payment through eSewa, where a booking is confirmed only after the payment is verified.
6. Reports for operators and administrators on bookings, revenue, and seat occupancy.

In addition, the project will deliver complete documentation, including the requirement analysis, design diagrams, and test results, and the source code managed in a Git repository. The system will be a working application, demonstrated with sample data.

5.2 Future Recommendations

The system can be extended in the following ways:

1. Live payments through eSewa with a registered merchant account, and other payment options.
2. Email and SMS notifications for booking confirmation and changes to a trip.
3. Live bus tracking using GPS.
4. A native mobile application.
5. Testing and tuning for a large number of simultaneous users.
6. Analytics that help operators decide on schedules and fares, such as demand and occupancy trends.

REFERENCES

- [1] BusSewa, “Nepal’s Trusted Online Ticketing & Travel Platform.” [Online]. Available: <https://bussewa.com.np/>. [Accessed: Oct. 4, 2026].
- [2] GoMyGo, “Book Bus Tickets Online in Nepal.” [Online]. Available: <https://gomygo.com/>. [Accessed: Oct. 4, 2026].
- [3] Okay Journey, “Nepal’s Leading Trusted Online Bus Ticket Platform.” [Online]. Available: <https://www.okayjourney.com/>. [Accessed: Oct. 4, 2026].
- [4] Trip Turbo, “How to Book Bus Tickets Online Using Trip Turbo in Nepal,” Mar. 9, 2025. [Online]. Available: <https://tripturbo.com/blog/how-to-book-bus-tickets-online/>. [Accessed: Oct. 4, 2026].
- [5] M. O. Oloyede, S. M. Alaya, and K. S. Adewole, “Development of an online bus ticket reservation system for a transportation service in Nigeria,” *Computer Engineering and Intelligent Systems*, vol. 5, no. 12, pp. 9–17, 2014.
- [6] M. Chavan *et al.*, “Online bus reservation system,” *Int. J. Sci. Res. Comput. Sci. Eng. Inf. Technol.*, vol. 7, no. 3, pp. 494–496, May–Jun. 2021.
- [7] Thotakura Jai Sai Venkat and A. Naga Raju, “A web-based online bus ticketing application for efficient public transport reservation and management,” *Int. J. Eng. Res. Sci. Technol.*, vol. 22, no. 2, pp. 1677–1688, Apr. 2026.
- [8] Z. Zhang, X. Zhang, and X. Li, “Securing high-concurrency ticket sales: A framework based on microservice,” arXiv:2512.24941, Dec. 2025. [Online]. Available: <https://arxiv.org/pdf/2512.24941>